

ERP Server Backend Architecture

A NestJS · Prisma · PostgreSQL platform for multi-company retail and distribution ERP — system design, data architecture, security posture, and engineering risk register.

77.7k	234	90	204
LINES OF TYPESCRIPT	HTTP ENDPOINTS	DATABASE MODELS	MIGRATIONS

Document	ERP Server — Backend Architecture Dossier
Revision	1.0
Status	Issued for internal review
Date of issue	14 July 2026
Applies to	Branch DEV-v1 @ 5b3b3f0 · 220 commits (13 Feb – 11 Jul 2026)
Repository	ERP server — erp-server@1.0.0
Audience	Engineering, architecture review board, security, operations
Classification	Confidential — internal use only. Contains unremediated security findings.

Sections 12 and 13 record open defects and security exposures identified during this review. They are reported as found, without mitigation. Distribution outside the engineering organisation is not authorised.

Contents

1	Executive summary	9	Transactional integrity and document numbering
2	System context and scope	10	Security architecture
3	Technology stack		10.1 Authentication · 10.2 Authorisation · 10.3 Input validation · 10.4 Audit trail
4	Runtime architecture	11	Caching, queues and background work
	4.1 Layering · 4.2 Request pipeline · 4.3 Bootstrap sequence	12	Observability and performance
5	Domain module catalogue	13	Build, packaging and deployment
	5.1 Map · 5.2 Endpoint distribution · 5.3 Module reference	14	Quality engineering
6	API design and contracts	15	Risk register
	6.1 URL grammar · 6.2 Envelopes · 6.3 Status mapping	16	Remediation roadmap
7	Data architecture	A	Appendix — configuration surface
	7.1 Schema composition · 7.2 Multi-schema model · 7.3 Conventions · 7.4 Migrations	B	Appendix — route index
8	The configured-SQL engine	C	Appendix — glossary

1 Executive summary

The ERP Server is a single-deployable NestJS monolith that provides the master-data, inventory, accounting and sales backbone for a multi-company, multi-branch retail ERP. It is a mature, internally consistent codebase: 77,701 lines of TypeScript across 69 controllers and 87 services, backed by 90 PostgreSQL models spread over eight schemas and 204 sequential migrations. Its defining architectural choice is that a large part of the user interface — grids, dropdowns, column layouts, and the SQL that feeds them — is *data*, not code.

1.1 What the system is

Functionally the platform covers the master-data and stock spine of a distribution ERP: an item catalogue with units, conversions, barcodes, taxes and price levels; a chart of accounts with a voucher engine; customers, suppliers and the geographic hierarchy above them; warehouses, opening stock, physical stock counts and a batch-level stock ledger; and the organisational structure — companies, branches, fiscal years, employees and users — that scopes all of it.

Two design decisions distinguish it from a conventional CRUD service. First, **customers and suppliers are account ledgers**: each one is provisioned with a row in `acc_ledger_master` and reuses that ledger's primary key as its own, so a customer *is* its ledger rather than pointing at one. Second, the **configured-SQL engine** stores the SELECT statement behind every grid and dropdown as a database row, parses and validates it at runtime, binds tenant parameters into it, and executes it on a read-only connection pool. Screens can be added or reshaped by an administrator without a deployment.

1.2 Engineering assessment

DIMENSION	RATING	BASIS
Structural consistency	Strong	55 modules follow one scaffold (DTO → filter → controller → service). A shared utility spine (<code>module-shared.utils.ts</code>) supplies pagination, error helpers and transaction typing to all of them.
Data integrity	Strong	294 interactive transactions; advisory-lock-guarded document numbering; universal soft delete; JSONB before/after audit snapshots written inside the business transaction.
Domain depth	Strong	The stock ledger implements batch identity resolution, two-tier balance roll-up and valuation as a genuine engine, not CRUD.
Security posture	Critical gaps	The entire Users controller is unauthenticated; role-based authorisation and the Prisma tenant-scoping extension were removed in commit <code>80ab8eb</code> and never replaced. See §10 and §15.
Operational readiness	Weak	No CI pipeline, no log rotation (168 MB <code>app.log</code>), no metrics or tracing, single-process PM2 deployment, and two queues with no producers.
Test coverage	Uneven	56 spec files, 37 of them in Inventory. Settings, purchase, fixed, users, cache and queue code have none.

PRINCIPAL FINDING

The application currently has **authentication but no authorisation**. Once a bearer token is issued, every one of the ~64 protected controllers is fully accessible, and no query is filtered by company or branch. A Prisma client extension that auto-scoped 35 models to the token's company was deleted in commit `80ab8eb` and has no replacement; a stale comment in `items-master.service.ts` still refers to it. Separately, `UserController` carries a class-level `@Public()`, which exposes unauthenticated user creation, user enumeration and password change. These are the two items that gate any production release.

1.3 How to read this document

Sections 2–9 describe the system as designed and built. Section 10 assesses the security architecture against what is actually enforced. Sections 11–14 cover the operational surface. Section 15 consolidates every defect and exposure found during the review into a single register with severity and location; section 16 sequences the fixes. Every claim in this document was verified against the source tree at commit `5b3b3f0`; file paths are given so that each can be re-checked.

2 System context and scope

2.1 Position in the wider system

The ERP Server is the sole backend for the ERP product. It is consumed by a browser-based administration and point-of-sale client over HTTP, and it owns the PostgreSQL database outright — there is no other writer. Redis is an optional dependency used for HTTP response caching, authentication sessions and BullMQ job transport; the application degrades to a no-op cache and boots successfully when Redis is absent.

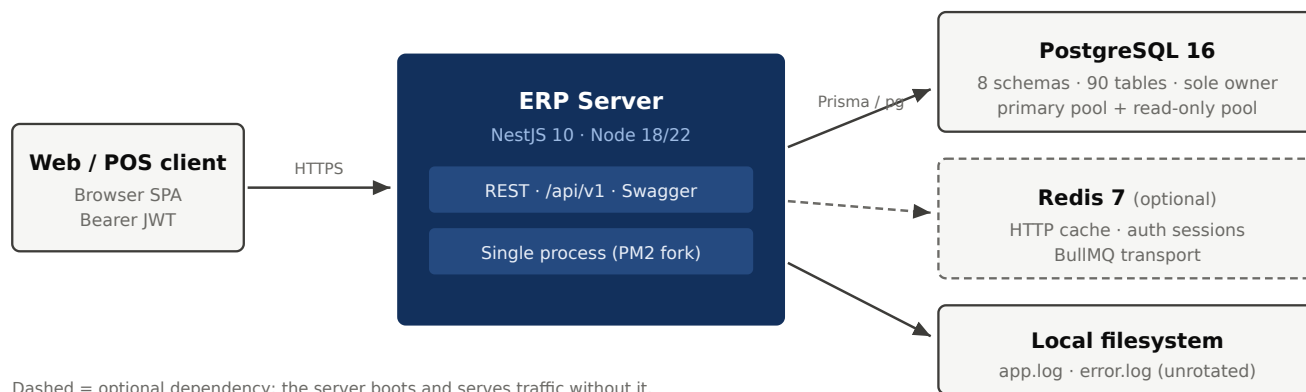


Figure 2.1 — System context. The server is the only writer to the database and the only integration point for the client.

2.2 In scope / out of scope

OWNED BY THIS SYSTEM	EXPLICITLY NOT PRESENT
Master data: items, units, taxes, prices, barcodes, godowns	Purchase and sales <i>transactions</i> (only quotations exist today)
Chart of accounts, ledgers, vouchers, e-invoice/e-way-bill document register	GST filing / GSP call-out (credentials are stored; no client implemented)
Stock: opening stock, physical stock, batch ledger, balances	Reporting/BI layer beyond the configured-grid engine
Organisation: companies, branches, fiscal years, employees, users	Message bus, external integrations, webhooks
Platform: audit trail, sequences, configurable grids and dropdowns	Horizontal scale-out (single process by configuration)

2.3 Quality attributes, as actually engineered

ATTRIBUTE	MECHANISM IN THE CODE
Consistency	Every write of consequence runs inside <code>prisma.\$transaction</code> ; the audit row is written with the same transaction client, so an audit entry cannot survive a rolled-back business write.
Recoverability	Nothing is hard-deleted. 80 of 90 models carry an <code>*_is_deleted</code> flag and delete endpoints toggle it.
Traceability	83 models carry <code>*_created_on / *_created_by</code> ; <code>audit.audit_log</code> stores JSONB before/after snapshots and a computed field-level diff.
Configurability	Grids, dropdowns, column widths, menus, batch prefixes and document-number formats are rows, not code.
Portability	The build produces a single self-contained executable per platform (<code>pkg</code>), targeting hosts with no Node runtime installed.
Availability	Redis failure degrades to a no-op cache rather than failing the boot; a database failure at startup is logged but not fatal.

3 Technology stack

RUNTIME AND FRAMEWORK

COMPONENT	VERSION	ROLE AND NOTES
Node.js	18 (target) / 22 (dev)	The <code>pkg</code> release targets <code>node18-win-x64</code> and <code>node18-linux-x64</code> .
TypeScript	5.7	Strict decorator metadata; <code>tsc --noEmit</code> available as <code>npm run typecheck</code> .
NestJS	10.4	Express adapter. URI versioning, global prefix, global guard/filter/interceptor chain.
Prisma ORM	6.19.2	Multi-schema (<code>@schema</code>) client. Interactive transactions are the dominant write pattern.
PostgreSQL	16 (alpine)	Eight active schemas. Requires a <code>uuidv7()</code> SQL function — 72 models depend on it for primary keys.
node-postgres (<code>pg</code>)	8.13	A second, deliberate access path: two pools, one forced <code>default_transaction_read_only=on</code> for user-authored SQL.
Redis	7 (alpine)	Optional. HTTP cache, auth session store, BullMQ transport.
BullMQ	5.76	Two queues registered — <code>audit-log-archival</code> and <code>stock-reconciliation</code> .

CROSS-CUTTING LIBRARIES

LIBRARY	APPLIED TO
<code>class-validator</code> / <code>class-transformer</code>	All 213 DTOs, through a shared decorator library (<code>src/common/dto/dtoDecorators.ts</code>). Enforced by a global <code>ValidationPipe</code> with <code>whitelist</code> and <code>forbidNonWhitelisted</code> .
<code>@nestjs/swagger</code>	One aggregate OpenAPI document plus ~60 per-module documents, generated from a registry at <code>src/utls/swaggerDocs.ts</code> .
<code>@nestjs/throttler</code>	Global rate limit, 100 requests / 60 s / IP, in-memory storage.
<code>helmet</code> , <code>compression</code>	Security headers (CSP relaxed for Swagger UI) and response compression.
<code>joi</code>	Boot-time validation of 40 environment variables (<code>src/config/env.validation.ts</code>).
<code>node:crypto</code>	Hand-rolled HS256 JWT and scrypt password hashing — <code>@nestjs/jwt</code> is a dependency but is <i>not</i> used.
<code>jest</code> / <code>supertest</code>	56 unit specs, 2 end-to-end specs.
<code>pkg</code>	Single-binary packaging, including the Prisma query engine and the <code>.env</code> file.

CONVENTION — NO ORM LOCK-IN ON THE READ PATH

Prisma owns every typed CRUD path. Raw SQL is confined to three places, by design: PostgreSQL advisory locks (inexpressible in the Prisma API), the `SELECT 1` liveness probe, and the configured-SQL engine, which by definition executes statements the ORM cannot know about. Nothing else reaches for `$queryRawUnsafe`.

4 Runtime architecture

4.1 Layering

The application is a layered monolith. Controllers are thin — they validate, delegate, and shape the response envelope. Services hold the domain logic and own transaction boundaries. Two persistence services sit beneath them, and a set of global cross-cutting components wraps every request from the outside.

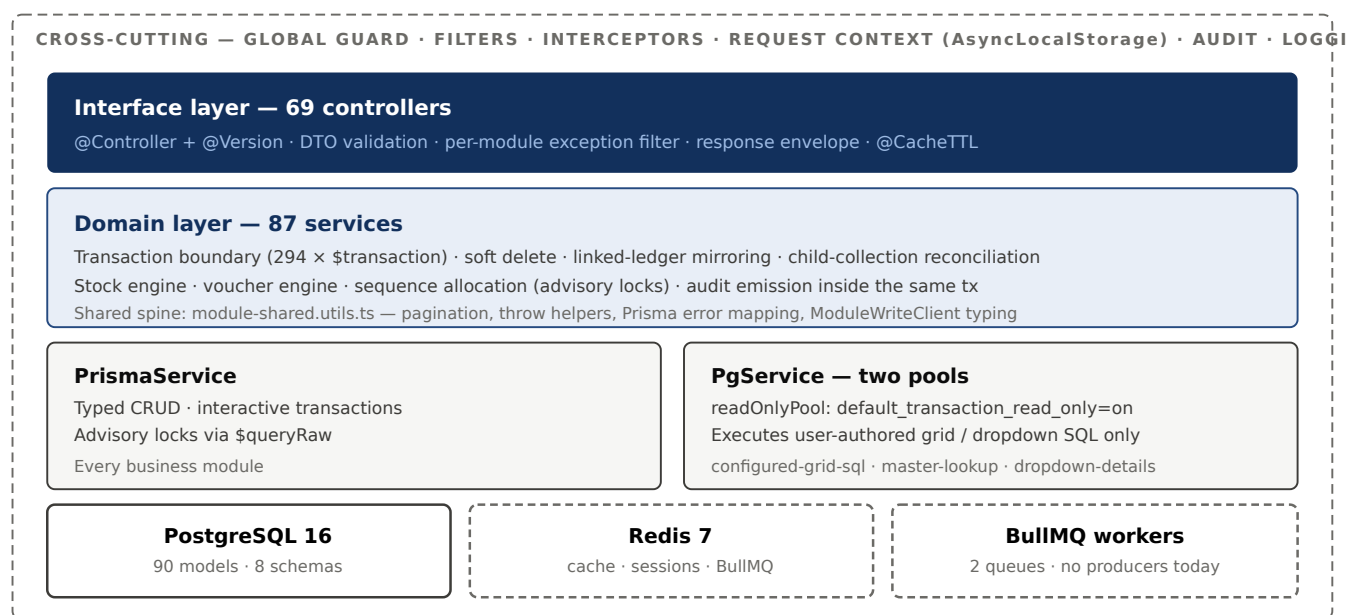


Figure 4.1 — Layering. The dual persistence path is intentional: PgService exists solely so that SQL stored in the database can be executed under a read-only session, isolated from the ORM.

4.2 The request pipeline

Every request passes through a fixed chain assembled in `src/main.ts` and `src/app.module.ts`. The order is load-bearing: body limits are registered before Nest initialises so they override the framework's 100 kB default, and the access-token guard runs before any interceptor so that request context is populated from the verified token — never from a header — before caching or handler logic sees the request.

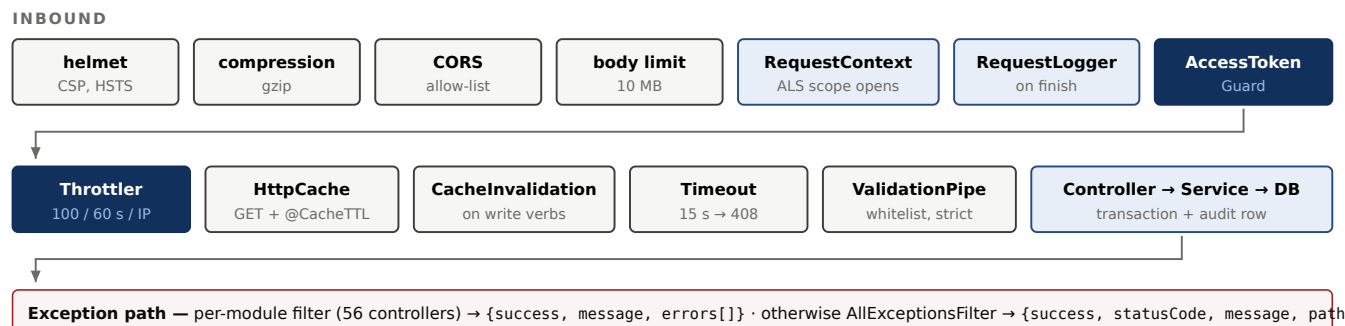


Figure 4.2 — The global request pipeline. Two divergent error envelopes leave the system depending on whether the controller declares a module filter (§6.2).

4.3 Bootstrap sequence

`src/main.ts` performs a deterministic 13-step boot. Two aspects are worth recording because they shape deployment. First, `src/env.preload.ts` is imported *before* `reflect-metadata` and before `AppModule`, so `process.env` is fully populated for module-evaluation-time reads — several modules (notably the throttler) read `process.env` directly rather than through `ConfigService`. Second, path resolution is `pkg`-aware: `resolveFilePath()` searches the working directory, then the executable's directory, then the packaged snapshot, so TLS certificates can sit beside the binary.

#	STEP	BEHAVIOUR
1	Env preload	<code>dotenv.config()</code> before any framework import.
2	HTTPS options	If <code>HTTPS_ENABLED</code> , cert and key are required — a missing path throws and aborts the boot.
3	Logger	<code>FileLoggerService</code> is constructed directly and passed to <code>NestFactory.create</code> , so the very first framework log line is already captured to file.
4	Shutdown hooks	<code>enableShutdownHooks()</code> — Prisma and both <code>pg</code> pools close on <code>onModuleDestroy</code> .
5– 7	Body limit → helmet → compression → CORS	In production, CORS is enabled <i>only</i> if <code>CORS_ORIGINS</code> is non-empty; otherwise it is never enabled at all.
8– 10	ValidationPipe, URI versioning, global prefix	Yields the URL shape <code>/api/v1/<resource></code> .
11	Swagger	Mounted unconditionally, including production. One aggregate document plus ~60 per-module documents.
12	Listen	<code>PORT</code> (3010 by configuration; PM2 overrides to 3012), <code>HOST</code> default <code>0.0.0.0</code> .
13	DB probe	<code>SELECT 1</code> . Failure is logged but does not abort the boot — the server will accept traffic with a dead database.

5 Domain module catalogue

Nineteen top-level modules decompose into 72 Nest modules and 69 controllers. Fifty-five of them are master-data modules built from the same four-file scaffold; the remainder are engines (stock, vouchers, sequences, configured SQL) or platform services (auth, audit, health, lookup).

5.1 Domain map

BUSINESS DOMAINS

Inventory 17 submodules · 47 routes items · units · taxes prices · barcodes · godowns composite item save (1 tx)	Sales 8 submodules · 30 routes customers · state/city/area quotations · loyalty · freight customer = ledger	Accounts 9 submodules · 26 routes COA · ledgers · vouchers tenders · GSP · e-invoice voucher-header helper	Stocks 4 submodules · 18 routes opening · physical count batch ledger · balances the stock engine	Purchase 2 submodules · 6 routes suppliers supplier groups supplier = ledger
---	--	---	--	---

ORGANISATION & REFERENCE

Settings — 24 routes companies · branches · employees user administration · configs	Fixed — 29 routes banks · HSN · state codes · menus UI tables · devices · price levels	Master — 10 routes batch prefixes dashboard widgets	Cust. rates — 4 routes negotiated per-customer item pricing overrides
--	---	--	--

PLATFORM SERVICES

Auth JWT + Redis session	Audit log 53 services emit	Configured SQL grids + dropdowns	Sequences document numbers	Master-lookup 8 read routes	Health DB + cache probe
------------------------------------	--------------------------------------	--	--------------------------------------	---------------------------------------	-----------------------------------

Figure 5.1 — Domain map. Business domains sit on a shared organisation/reference tier and a platform tier that the whole system draws on.

5.2 Endpoint distribution

The 234 route decorators are unevenly distributed, and the shape is diagnostic: Inventory alone accounts for one route in five, and the top four modules carry 56% of the surface. This concentration is why Inventory is also where almost all the test coverage sits (§14).

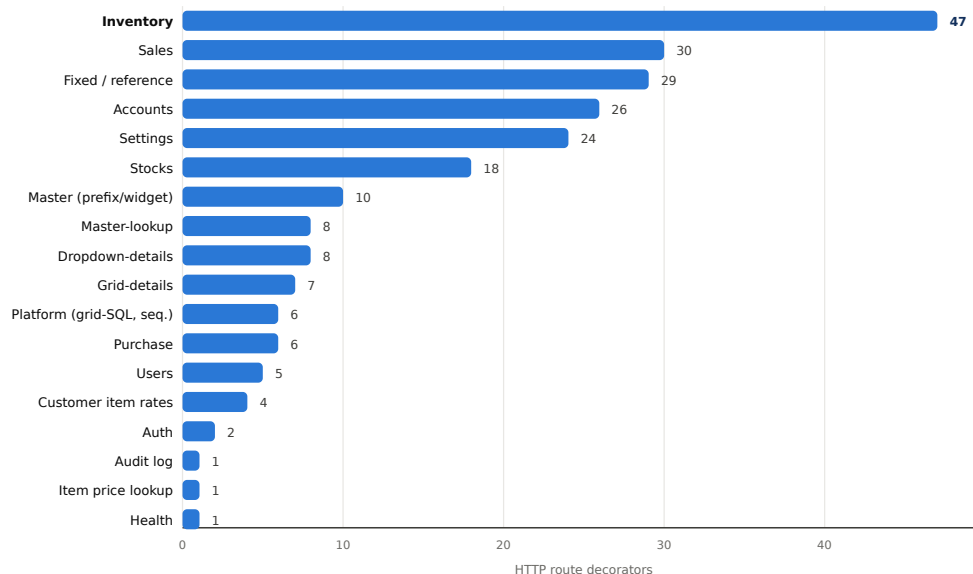


Figure 5.2 — Route decorators per module (n = 233; one further route belongs to an orphaned, unregistered duplicate module). Counts include alias routes, so they exceed the number of distinct business operations.

5.3 Module reference

MODULE	ROUTES	RESPONSIBILITY AND THE PATTERN THAT DEFINES IT
Inventory	47	Item catalogue and its satellites. <code>ItemsMasterService.saveComposite</code> writes the item and its unit conversions, prices, barcodes and reorder levels in one transaction; <code>item-master-update.service.ts</code> reconciles child collections by <i>natural key</i> , writing only genuine deltas.
Sales	30	Customers, the state→city→area hierarchy, quotations, freight slabs, loyalty. A customer's primary key <i>is</i> its account-ledger id, and its area id becomes the ledger's parent account group.
Fixed	29	Reference data (banks, HSN, state codes, devices, price levels) plus UI scaffolding (menu tree, UI table definitions) and login-session records.
Accounts	26	Chart of accounts, ledgers, tender types, GSP registration. <code>accountVoucherHeader</code> has no controller — it is a pure helper that lets stock documents participate in accounting.
Settings	24	Companies, branches, employees, user administration, key–value configs, GSP service credentials.
Stocks	18	The movement engine. <code>item-stock-ledger.service.ts</code> (1,700 lines) resolves batch identity, auto-generates batch numbers from a configured prefix, applies movements to batch and summary rows, and rolls batches up into the item summary.
Master-lookup	8	One read API feeding every dropdown in the UI, via a fetcher registry keyed by master name, with fallback to administrator-configured SQL.
Grid / dropdown details	15	The metadata engines: stored SQL, column widths, filters, visibility — persisted per screen (§8).
Purchase	6	Suppliers and supplier groups, mirrored into account ledgers exactly as customers are.
Auth · Audit · Health	4	Login/refresh; the audit query API; the DB + cache probe.

HOUSEKEEPING — DEAD CODE

`src/modules/item-price-details/` is an orphaned duplicate of `src/modules/Inventory/item-price-details/`. It has no `.module.ts`, is imported by nothing, and declares the same `@Controller('item-price-details')` prefix as the live one. It is not routable and should be deleted. Similarly, `src/modules/stocks/itembatchstock/` and `itemstockbalance/` use camelCase filenames (`itemBatchStockController.ts`) against the kebab-case convention held by the other 67 controllers.

6 API design and contracts

6.1 URL grammar

All routes are versioned by URI under a configurable prefix:

`{/API_PREFIX}/v{API_VERSION}/{resource}/{action}`, in practice `/api/v1/...`. The house style is **verb-in-path CRUD** rather than resource REST, and it is applied with near-total consistency across the master modules:

```
POST /api/v1/item-groups/create → upsert: creates when no id is supplied, updates when one
is
GET /api/v1/item-groups/get → single record by id (query param)
GET /api/v1/item-groups/list → paginated list: ?search=&page=&limit=
DELETE /api/v1/item-groups/delete → toggles the soft-delete flag; returns { id, deleted:
boolean }
```

Across the codebase this yields `get` ×64, `delete` ×54, `create` ×51 and `list` ×10 as the dominant sub-paths. The trade-off is deliberate but should be stated plainly: the API is not idiomatic REST, so HTTP caching, conditional requests and standard client tooling gain little from the method semantics. Two controllers (`users`, `physical-stock`) additionally expose `:id`-style routes as aliases, which is the only place the two styles collide.

6.2 Response envelopes

Successful responses are assembled by hand in each controller — there is no global response interceptor — against a shared type, `ModuleApiResponse`:

```
{
  "success": true,
  "message": "Item groups fetched successfully",
  "data": [ ... ],
  "meta": { "page": 1, "limit": 20, "total": 3, "total_pages": 1 },
  "styles": [ ... ] // optional: grid column styling, when served by the configured-SQL
engine
}
```

Two endpoints deliberately break the envelope: `GET /health` returns a bare status object, and `POST /auth/login|refresh` return a bare token payload.

DEFECT — THE ERROR CONTRACT IS NOT UNIFORM

There are **two** error envelopes, and which one a client receives depends on whether the controller declares a module-scoped exception filter. Fifty-six controllers do, and return

`{success:false, message, errors:[{field, message}]}` — with no status code, path or timestamp in the body. The remaining thirteen — including `auth`, `health` and `users` — fall through to the global `AllExceptionsFilter` and return `{success:false, statusCode, message, path, timestamp}`, where `message` may be a string, an array of strings, or a nested object. A client cannot parse errors uniformly.

Consolidating on one envelope is a low-risk, high-value fix (§16).

A further subtlety worth knowing when debugging: the module filter derives the `field` name in each error by running a *regular expression* over the validation message (each module supplies its own field-name pattern, e.g.

`/\b(arm[A-Za-z0-9]+)\b/` for Area). When no match is found the field is reported as the literal string `"request"`.

6.3 Status-code mapping

STATUS	RAISED BY	CONDITION
400	Global <code>ValidationPipe</code>	DTO failure, or any property not on the DTO (<code>forbidNonWhitelisted</code>).
400	<code>throwBadRequest()</code>	Business-rule violation; also the mapped result of a SQLSTATE class-42 error from configured SQL (undefined column/table is treated as a configuration fault, not a server fault).
401	<code>AccessTokenGuard</code>	Missing or malformed <code>Bearer</code> header, invalid signature, expired token, or a session that is no longer active in Redis.
404	<code>throwNotFound()</code>	Record absent or soft-deleted.
408	<code>TimeoutInterceptor</code>	Handler exceeded the hard-coded 15,000 ms budget.
409	<code>throwConflict()</code>	Prisma <code>P2002</code> unique-constraint violation, mapped centrally by <code>throwOnUniqueConstraintError()</code> .
429	<code>ThrottlerGuard</code>	More than <code>THROTTLE_LIMIT</code> requests per <code>THROTTLE_TTL</code> from one IP.
503	<code>HealthController</code>	Database or cache probe failed.
500	Either filter	Anything else. Stack traces are logged server-side only, never returned.

CONVENTION — NULL VERSUS UNDEFINED IN DTOS

The shared decorator library encodes a three-state update semantics that is easy to miss and important to preserve: `undefined` means *do not touch this column*, while an explicit `null` means *clear it*. This is implemented by `SkipOnNullish()` — a `ValidateIf` that suppresses validation only for nullish values — wrapped into every `Nullable*` decorator. Partial updates therefore work without a separate PATCH DTO.

7 Data architecture

Ninety models across eight PostgreSQL schemas, generated from 92 hand-maintained schema fragments and evolved through 204 sequential migrations. The schema is master-data dominated — roughly 75 of the 90 models are masters or configuration — with transactional depth concentrated in the voucher stack, the stock ledger and quotations.

7.1 Schema composition — the generated schema is not the source of truth

`prisma/schema.prisma` (4,803 lines) is a build artefact. It is regenerated by `scripts/build-prisma-schema.js`, which recursively walks `prisma/`, concatenates `base.prisma` with every other fragment in path-alphabetical order, and stamps each with a provenance comment. Editing the generated file is futile — it is rebuilt before every Prisma command, including `postinstall`.

```
// AUTO-GENERATED FILE. DO NOT EDIT.
// Generated by scripts/build-prisma-schema.js

// ----- prisma/inventory/itemMaster.prisma -----
model ItemMaster { ... }
```

The pipeline enforces one invariant with a hard failure: only `base.prisma` may declare a `generator` or `datasource` block. A fragment that tries to raises an error and aborts the build. This keeps the `datasource` — and the 12-schema declaration — in exactly one place.

CONVENTION — WHERE TO MAKE A SCHEMA CHANGE

Edit the fragment under `prisma/<domain>/*.prisma`, never `prisma/schema.prisma`. Then run `npm run prisma:migrate:dev`, which rebuilds the schema before invoking Prisma. Every Prisma npm script is wired this way, so a stale generated schema is structurally impossible.

7.2 The multi-schema model

Twelve PostgreSQL schemas are declared in the `datasource`; eight are actually populated. Every model carries an explicit `@@schema(...)`. Note that the folder layout does not map one-to-one onto schemas — `prisma/dropdown/` models are annotated `@@schema("fixed")`, and `prisma/stocks/` maps to the singular `stock`.

SCHEMA	MODELS	PRINCIPAL ENTITIES
<code>inventory</code>	22	ItemMaster, ItemGroupMaster, categoryMaster, ItemBrandMaster, ItemSectionMaster, Unit, ItemUnitConversion, ItemEanCode, ItemBatchMaster, ItemBatchStock, ItemStockBalance, ItemStockLedger, ItemPriceMaster, ItemQtywiseRate, ItemReorder, ItemTaxMaster, ItemTaxHistory, GodownLocation, OpeningStockHeader/Detail
<code>accounts</code>	17	AccountGroup, AccLedgeMaster, AccLedgeBankAccount, AccShipAddr, AccVoucher, AccVoucherHeader, AccVoucherType, AccVoucherSeq, AccVoucherBill/Cheque, AccVoucherDocRegister/Detail/Einvoice/Ewaybill/ApiLog, AccountTenderMaster/Types
<code>sales</code>	17	Customer, CustGroup, CustItemRate, StateMaster, CityMaster, AreaMaster, EmployeeMaster/Department/Designation, SaleQuotation(+Item), FreightCharges, LoyaltyScheme(+Point/Gift/Party)
<code>fixed</code>	17	GridDetails, GridColumn, DropdownDetails, DropdownColumns, Uitable, UitableColumns, FormSection, FormField, Menu, BankMaster, DeviceMaster, StateCode, HsnMaster, PriceLevel, StockAdjReason, GspProviderMaster, GspCompanyService
<code>public</code>	9	Company, BranchMaster, CompanyGroupMaster, UserMaster, UserMenus, FiscalYear, BatchPrefix, Configs, AppThemeMaster
<code>audit</code>	3	AuditLog, AuditScreen, UserLoginSession

SCHEMA	MODELS	PRINCIPAL ENTITIES
stock	3	PhysicalStockHeader, PhysicalStockDetail, PhysicalStockBatchDetail
purchase	2	Supplier, SupplierGroup
Declared but unused: <code>grid</code> , <code>settings</code> , <code>reports</code> , <code>transport</code> . Note the grid engine lives in <code>fixed</code> , not <code>grid</code> .		

7.3 Modelling conventions

The schema is written in a legacy-ERP idiom that is applied uniformly. Understanding these six conventions is enough to read any table in the system.

CONVENTION	RULE, AND HOW CONSISTENTLY IT HOLDS
Column prefixes	Every column is prefixed with a per-table abbreviation — <code>item_*</code> , <code>avh_*</code> , <code>cus_*</code> , <code>sup_*</code> , <code>grid_*</code> , <code>seq_*</code> . Prisma fields are camelCase, mapped with an explicit <code>@map</code> to snake_case. This is also why configured-SQL parameters are prefixed <code>p_*</code> — so a parameter token can never be mistaken for a real column.
Primary keys	72 models use <code>@default(dbgenerated("uuidv7()")) @db.Uuid</code> — time-ordered UUIDs, validated at the edge with <code>ParseUUIDPipe({version:'7'})</code> . This requires a <code>uuidv7()</code> function to exist in the database; it is a hard deployment prerequisite. Fourteen small lookup tables use <code>autoincrement()</code> .
Soft delete	80 of 90 models carry an <code>*_is_deleted</code> boolean. Enforcement is <i>application-level</i> : there is no Prisma middleware or extension applying it, so every read must filter explicitly. Frequently paired with an independent <code>*_is_active</code> flag (68 models) — a two-axis lifecycle where <i>deleted</i> means invisible and <i>inactive</i> means visible but unusable.
Audit columns	<code>*_created_on</code> (83 models), <code>*_created_by</code> (82), <code>*_modified_on</code> (58), <code>*_modified_by</code> (56), all <code>Timestampz(6)</code> . The actor is not injected by middleware — services read it from <code>RequestContextService.getUserId()</code> and write it explicitly, falling back to a sentinel <code>DEFAULT_ACTOR</code> UUID of all zeroes when no user is present.
Tenancy columns	34 models carry <code>*_company_id</code> and 36 carry <code>*_branch_id</code> ; branch is usually nullable, meaning "shared across branches". Composite indexes follow the scope, e.g. <code>@@index([companyId, branchId, accYear])</code> .
Linked ledger	Customers and suppliers share a primary key with an <code>acc_ledger_master</code> row: the ledger is created first inside the transaction and its <code>led_id</code> becomes the master's own id. Field propagation is table-driven by a copy-map; updates and soft-deletes are mirrored. This is the single most important non-obvious invariant in the schema.

CRITICAL — TENANT ISOLATION IS NOT ENFORCED ANYWHERE

The tenancy columns exist, but nothing applies them. A Prisma client extension (`src/database/prisma/tenant-scope.extension.ts`) that auto-scoped 35 models to `requestContext.getCompanyId()` on every `findMany`, `findFirst`, `count`, `aggregate`, `groupBy`, `updateMany` and `deleteMany` was **deleted in commit 80ab8eb** and never replaced. There is no row-level security in the database either. Isolation now depends entirely on each service — and each stored `grid_sql` — remembering to apply the predicate, and most services do not. A stale comment in `items-master.service.ts:602` still tells the reader that "reads are auto-scoped to the token's company (see `tenant-scope.extension.ts`)". They are not. See R2, §15.

7.4 Migration strategy

204 migrations run from `20260212000000_init` to `20260713072139_...` — roughly 1.3 migrations per calendar day over five months, which is the signature of a schema still being actively discovered rather than maintained. Deployment uses `prisma migrate deploy` (aliased as `npm run migration:run`).

Migrations are not only DDL. Several rewrite rows in `fixed.grid_details.grid_sql` to schema-qualify table names after models moved between schemas. That practice — editing data with migrations, then correcting the migration in place — produced the need for two bespoke repair scripts, both of which are worth understanding before operating this system:

SCRIPT	PROBLEM IT EXISTS TO SOLVE
<code>sync-prisma-migration-checksums.js</code>	Two applied migrations were later <i>edited in place</i> , so their SHA-256 no longer matches the value recorded in <code>_prisma_migrations</code> , and Prisma refuses to deploy. The script re-hashes the files and updates the stored checksums inside a transaction (<code>--dry-run</code> supported). It is a deliberate escape hatch that <i>blesses drift</i> — the safer convention, which the repo also follows elsewhere, is a forward-only fix migration.
<code>repair-ui-table-drift.js</code>	A historical rename of <code>fixed.ui_table</code> → <code>fixed.ui_table_columns</code> left older databases in a state Prisma could only fix by a destructive reset. The script performs a non-destructive, transactional repair: rename the table, rename and re-own the identity sequence, restore the column default, re-add the missing FK column, then add the constraint <code>NOT VALID</code> and only <code>VALIDATE</code> it once an orphan check passes. It refuses to guess if both tables exist.

Two generators run on every `prisma generate`: `prisma-dbml-generator`, which writes an importable ER model to `dbml/awesome.dbml` (90 tables — this is the artefact to load into dbdiagram.io), and `prisma-docs-generator`. The latter's output path, `output = "../.. / docs"`, resolves *outside the repository* to the parent workspace directory. That is almost certainly unintended and should be corrected.

8 The configured-SQL engine

This is the system's most distinctive subsystem and the one most worth guarding. Grids and dropdowns — their columns, widths, alignment, conditional colouring, totals, and the `SELECT` that populates them — are rows in `fixed.grid_details` and `fixed.dropdown_details`. The server parses SQL it did not write, validates it, binds tenant parameters into it, and executes it on a read-only connection. A new screen is a database row, not a release.

8.1 Execution pipeline

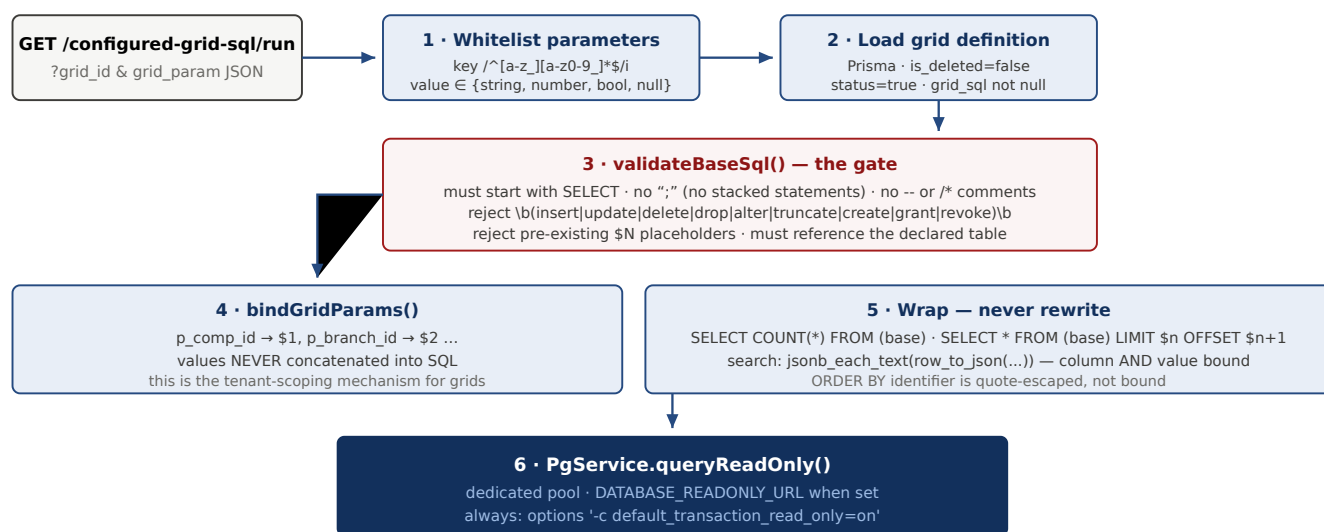


Figure 8.1 — Defence in depth: a syntactic deny-list, strict parameter binding, and a PostgreSQL session that physically cannot write. A statement that slips past the regex still cannot mutate data.

8.2 Parameter binding

The convention, discovered from the stored rows themselves, is that configured SQL embeds parameters as bare `p_`-prefixed identifiers. `bindGridParams` rewrites each token to a PostgreSQL positional parameter and returns the values separately, so nothing is ever concatenated:

```
// stored in fixed.grid_details.grid_sql
SELECT ls_item_name, ls_qty FROM inventory.item_stock_balance
WHERE ls_comp_id = p_comp_id AND ls_branch_id = p_branch_id

// grid_param = {"p_comp_id":"...", "p_branch_id":"..."} ⇒ bound form:
SELECT ... WHERE ls_comp_id = $1 AND ls_branch_id = $2      params = [comp, branch]
```

Binding order is deterministic (parameter key order); a token appearing many times binds to the same `$N`; and a key that never appears in the SQL contributes no parameter at all, so an unused parameter is harmless rather than a syntax error.

HIGH — THE DROPDOWN PATH WAS NEVER MIGRATED TO BINDING

The predecessor of `bindGridParams` is `substituteGridPrm()`, which *string-concatenates* escaped literals into the SQL. It is still live, called from `dropdown-details.service.ts:118` with client-supplied parameters. The classic escape is closed — keys are whitelisted by regex, values are restricted to scalars, and single quotes are doubled — but this is manual quoting, not binding, and it will misbehave if a token lands inside a dollar-quoted string or an identifier position. The grid path was fixed; the dropdown path is a regression waiting to be found. Separately, `master-lookup.service.ts:859` executes stored dropdown SQL with *no parameters at all* and swallows failures in a `try/catch { continue }`. See R3, §15.

8.3 Why there is a SQL parser in the codebase

Because the engine must reason about statements it did not author, `configured-grid-sql.service.ts` contains a small, hand-written, quote- and paren-aware SQL analyser — `extractTopLevelSelectClause`, `splitTopLevelCommaSeparated`, `extractSqlOutputFieldName`, `extractTopLevelFromRelation`, `stripSqlComments`. It exists to answer two questions: which output columns does this statement produce (so that `grid_columns` rows can be matched to them, by explicit mapping or by a scored heuristic), and which relation does it read from (so the stored SQL can be checked against the grid's declared table). This is inherent complexity of the data-driven design, not accidental complexity — but it is the part of the system that most rewards careful test coverage, and it currently has two spec files.

9 Transactional integrity and document numbering

9.1 Transaction discipline

Interactive Prisma transactions are the dominant write pattern — 294 occurrences across roughly 60 service files. The discipline that makes them compose is a single typing convention: services accept an optional transaction client, so an aggregate write can pull child services into its own transaction.

```
export type ModuleWriteClient = Prisma.TransactionClient | PrismaService;

// a child service participates in the parent's transaction ...
async save(dto: SaveItemPriceDto, tx?: Prisma.TransactionClient) { ... }

// ... so the composite item save is genuinely atomic
await this.prisma.$transaction(async (tx) => {
  const item = await this.writeItem(dto, tx);
  await this.itemUnitConversionService.syncChildren(item.itemId, dto.unit_conversions, tx);
  await this.itemsPriceMasterService.syncChildren(item.itemId, dto.prices, tx);
  await this.auditLogService.logEntityChange({ ... }, tx); // audit commits with the business
write
});
```

Three consequences follow, and all three are load-bearing. An audit row cannot survive a rolled-back business write, because it is written with the same client. A customer cannot exist without its ledger, because both are created in one transaction. And the child-collection reconciler in `item-master-update.service.ts` can diff incoming rows against existing ones by natural key — `ean_code`, `iuc_unit_id`, `ipm_unit_id + ipm_godown_id` — and write only genuine deltas, without risking a partially-applied item.

9.2 Document numbering under concurrency

ERP document numbers cannot be PostgreSQL sequences: they must reset per fiscal year, be gap-free within a scope, and vary by branch and POS device. The system therefore allocates them with a **transaction-scoped advisory lock plus $\max(n)+1$** , inside the write transaction:

```
await client.$queryRaw`
  WITH advisory_lock AS (
    SELECT pg_advisory_xact_lock(
      hashtext(${SEQUENCE_LAST_NO_LOCK_NAMESPACE}),
      hashtext(${this.buildLastNoScopeKey(scope)}) // company|branch|accYear|deviceId
    )
  ) SELECT 1::int AS locked`;
```

The lock is namespaced and released automatically at commit or rollback — no cleanup path can leak it. The counter itself lives in `accounts.acc_voucher_seq`, whose uniqueness constraint is the numbering matrix:

```
@@unique([vchrTypeId, companyId, branchId, accYear, deviceCode, periodKey],
  map: "uq_acc_voucher_seq_scope")
// document type × company × branch × fiscal year × device × reset period
```

The row also snapshots the applied format (prefix, company code, branch code, zero-pad width, suffix) and caches the last rendered reference. At write time `account-voucher-header.helper.ts` allocates three numbers per voucher —

an internal voucher number, a per-type serial, and the human-readable reference built from them — and re-numbers only when the scope actually changes, so an edit that does not move a voucher across company, branch or year keeps its number. Stock batches use the same idea at a different granularity: `generateNextBatchNo()` draws its prefix from the `batch-prefix` master and scopes batch identity by company, branch, godown, item and unit.

CONVENTION — NUMBERING IS NEVER A DATABASE SEQUENCE

If you add a new numbered document, do not reach for `SERIAL`. Follow the existing pattern: acquire `pg_advisory_xact_lock(hashtext(namespace), hashtext(scope))` inside the transaction, take `max(n)+1` within the scope, and format from the snapshot columns. A database sequence cannot be reset per year or made gap-free per branch, and mixing the two strategies would silently break the guarantee the ERP relies on.

10 Security architecture

This section describes what is *enforced*, not what is intended. The authentication layer is carefully built — hand-rolled HS256 tokens with strict verification, scrypt password hashing with constant-time comparison, and Redis-backed session revocation. The authorisation layer, by contrast, does not exist: it was removed and not replaced. That asymmetry is the defining fact of the system's current security posture.

10.1 Authentication

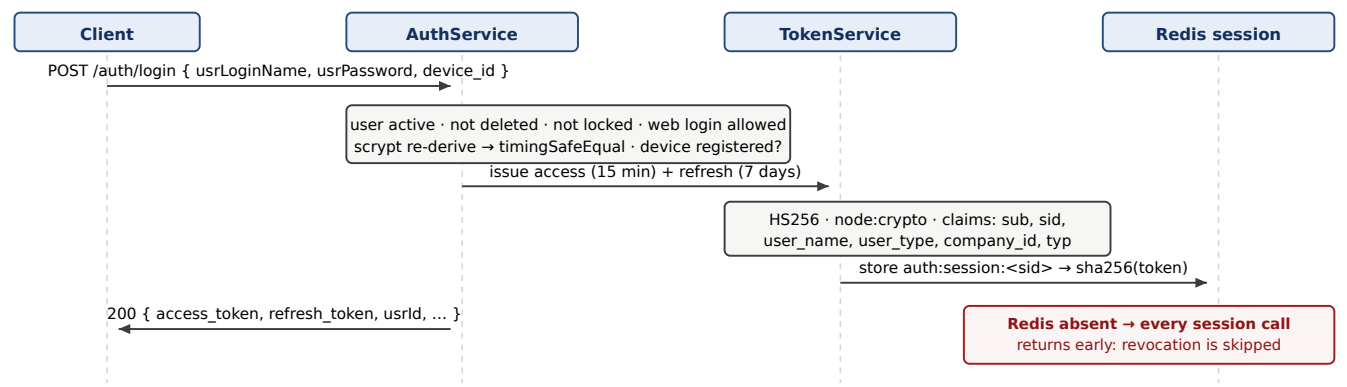


Figure 10.1 — Login. Every subsequent request re-verifies the signature *and* asserts the session is still active — unless Redis is disabled or down, in which case the assertion is silently skipped.

CONTROL	IMPLEMENTATION
Password storage	scrypt (node:crypto), 16-byte salt, 64-byte key, stored as scrypt\${salt}\${hash} . Verification re-derives and compares with timingSafeEqual . Default cost parameters (N=16384, r=8, p=1); no configurable cost and no versioned upgrade path.
Token format	Hand-rolled HS256 JWT. @nestjs/jwt is a dependency but unused. Verification rejects alg:none by requiring an exact {alg:'HS256', typ:'JWT'} header, compares the signature with timingSafeEqual , and enforces a custom typ claim (access vs refresh) that prevents token-type confusion. No iss , aud , nbf or jti .
Token lifetime	Access 900 s; refresh 604,800 s. Refresh re-reads the user, so a disabled account loses access at the next refresh — but refresh tokens are not rotated , and the same token is returned each time.
Session / revocation	Redis key auth:session:<sid> holding SHA-256 hashes of both tokens. revokeAccessTokenSession() exists but no route calls it — there is no logout endpoint. Sessions are written with no TTL, so the key space grows without bound.
Device binding	Login resolves the supplied device_id against the device master; an unregistered, blocked or inactive device is rejected.
Lockout	Not implemented. usrFailedLoginCount is incremented on every failure, but no threshold ever sets usrIsLocked — grep confirms the column is never written anywhere in src/ . Likewise usrMustChangePassword and usrPasswordExpiresOn are stored but never enforced at login.

10.2 Authorisation — the gap

AccessTokenGuard is registered globally as APP_GUARD , so by default every route requires a valid bearer token. Exemption is by the @Public() decorator. That is the entirety of the access-control model. There is no role check, no permission check, and no tenant filter behind it.

WHAT EXISTS	WHAT IS ENFORCED
UserType enum: SUPER ADMIN, ADMIN, MANAGER, USER, CASHIER, VIEWER, SYSTEM	Nothing. It survives as a JWT claim and a response field. @Roles() and RolesGuard were deleted in commit 80ab8eb .
Per-menu permissions umCanView/Create/Edit/Delete/Print/Export	Stored, and returned to the client so it can hide buttons. Never checked server-side. The UI is the only enforcement point.

WHAT EXISTS	WHAT IS ENFORCED
Company/branch columns on 34–36 models	Nothing. The Prisma tenant-scope extension was deleted with the same commit. <code>getCompanyId()</code> is now referenced in exactly one file, on a write path.

CRITICAL — THREE COMPOUNDING EXPOSURES

(1) The Users controller is public. `src/modules/users/users.controller.ts:36` carries a class-level `@Public()`. That exposes, with no token at all: `POST /users` (create a user with a chosen password), `GET /users` (enumerate all users), `PATCH /users/:id` (change any user's password) and `DELETE /users/delete`. This is a full authentication bypass for the whole application — an attacker creates a user, then logs in normally.

(2) Any authenticated user is fully authorised on the remaining ~64 controllers, across all tenants.

(3) Any authenticated user can author server-executed SQL via `POST /grid-details/create` or `POST /dropdown-details/create`, then run it. The read-only pool and the deny-list confine it to `SELECT`, but a `SELECT` over `user_master.user_password_hash` or any other tenant's data satisfies both. If `DATABASE_READONLY_URL` is unset, that statement runs as the primary application role.

10.3 Input validation and transport

Validation is strict and global: `whitelist: true` strips unknown properties and `forbidNonWhitelisted: true` turns them into a 400, so mass-assignment is structurally prevented. All 213 DTOs are built from the shared decorator library, which keeps `@MaxLength`, trimming and UUID-version checks uniform. Two weaknesses stand out:

`CreateUserDto.user_password` carries `@IsString()` only — no minimum length, no complexity, no maximum (an unbounded input to `bcrypt`) — and `LoginAuthDto.ip_address` is accepted from the request body and written to the login-session row in preference to the server-observed IP, making session audit IPs attacker-controlled.

TRANSPORT CONTROL	SETTING
Helmet	Enabled with a custom CSP. <code>scriptSrc</code> includes <code>'unsafe-inline'</code> (required by the Swagger UI customisation) — globally , not just on the docs route. <code>upgradeInsecureRequests</code> is explicitly disabled so Swagger works over plain HTTP.
CORS	Allow-list from <code>CORS_ORIGINS</code> ; localhost origins are added automatically outside production. <code>*</code> reflects any origin. Nothing prevents the combination <code>CORS_ORIGINS=*</code> with <code>CORS_CREDENTIALS=true</code> .
Rate limiting	100 requests / 60 s / IP, in-memory. No login-specific throttle — no <code>@Throttle()</code> anywhere — so with no lockout, roughly 100 password guesses per minute per IP are permitted. In-memory storage also means the limit is per-process and would not hold across a clustered deployment.
HTTPS	Optional, via <code>HTTPS_ENABLED</code> + cert/key paths, resolved <code>pkg</code> -aware so certificates can sit beside the binary.
Swagger	Served unconditionally, including in production , with no auth gate.

10.4 The audit trail

Auditing is explicit, not interceptor-driven: 53 services inject `AuditLogService` and call `logEntityChange(input, tx)` inside their transaction. Coverage is therefore a matter of developer discipline — a service that forgets the call writes no audit row, and there is no audit of reads or of authentication events.

What is captured is genuinely rich: action (`insert|update|approve|cancel`), screen, table, primary key, entity id, JSONB *before* and *after* snapshots plus a computed field-level diff, actor, branch, and client IP. The `audit-screen-sql.constants.ts` templates are not executed — they are persisted to `audit_screen` and parsed to derive source-field → human-label mappings for the read API, which also resolves foreign-key UUIDs back to display names.

HIGH — THE AUDIT READ API IS UNSCOPED

`GET /api/v1/audit-logs/list` requires a bearer token but applies **no company, branch or user filter**. Any authenticated user can read every audit row for every tenant — including the JSONB before/after snapshots, which by design contain whatever the audited row contained: PAN and Aadhaar numbers, GSTIN, bank account numbers and IFSC codes, and employee salary fields. This turns the audit trail into the single richest data-exfiltration target in the system. See R4, §15.

11 Caching, queues and background work

11.1 HTTP response cache

Two global interceptors implement a read-through cache with write-invalidation. `HttpCacheInterceptor` caches a `GET` only when the handler carries an explicit `@CacheTTL(n > 0)`; the key is the pathname plus a *sorted* query string, so `?b=2&a=1` and `?a=1&b=2` share an entry. It sets an `X-Cache: HIT|MISS` header on every cacheable response, and it swallows all cache errors — a Redis failure degrades to a pass-through, never a 500. `HttpCacheInvalidationInterceptor` then clears `/{{prefix}}/{{version}}/{{resource}}*` after any write verb, plus a small table of related resources.

TWO CACHE DEFECTS WORTH FIXING TOGETHER

No tenant dimension in the key. The cache key is URL + query only. Any cached `GET` is therefore shared across *all* callers regardless of who they are or which company they belong to. Today the blast radius is limited by the short TTLs below and by the fact that responses are largely tenant-agnostic masters — but it is a cross-tenant data leak waiting for the first company-scoped cached endpoint. (R7)

The default TTL is 3,600× too short. `redis.module.ts` computes `const ttlMilliseconds = ttlSeconds * 1;` — evidently intended as `* 1000` — and hands the result to a store that expects milliseconds. With `REDIS_TTL=3600` the default cache TTL is 3.6 seconds, not one hour. (R8)

Compounding this, roughly 50 controllers declare `@CacheTTL(1)` — one second — and about twelve use `@CacheTTL(60)`. Comments in `configured-grid-sql.controller.ts` claim "cache 5 minutes" and "cache 60s" beside a literal `1`. In practice the HTTP cache is a one-second de-duplication window for most master endpoints, which is close to no cache at all. The invalidation service also uses Redis `KEYS` (O(N), blocking) rather than `SCAN`, once per write request.

11.2 Queues

QUEUE	JOB AND BEHAVIOUR
<code>audit-log-archival</code>	Deletes audit rows older than <code>olderThanDays</code> (default 365) in batches of 1,000, reporting progress. Retries 3× with exponential backoff.
<code>stock-reconciliation</code>	For a company/year scope, compares each <code>item_batch_stock</code> closing quantity against the summed stock ledger and returns the mismatches. Retries 2×. It <i>detects</i> drift but never repairs it, and it issues one aggregate query per batch row — an N+1 by construction.

DORMANT INFRASTRUCTURE

Both queues are registered and both workers run, but **nothing enqueues a job**. `QueueService` has no callers outside its own directory; there is no scheduler (`@nestjs/schedule` is not a dependency) and no controller triggers either job. The archival and reconciliation capabilities exist structurally but have never executed in production. Either wire a schedule to them or remove them — dormant code that appears to provide a guarantee is worse than absent code. (R14)

11.3 Redis client

Worth flagging for maintainers: `RedisCacheService` is a **hand-rolled RESP protocol client** written directly on `node:net` / `node:tls` — it encodes commands and parses `+ - : $ *` replies itself — even though the `redis` package is already a dependency. It handles reconnection and pending-response queueing, and it is used by the health probe and the auth session store. It has no test coverage.

12 Observability and performance

12.1 Logging

`FileLoggerService` extends Nest's `ConsoleLogger` and appends to two files: `logs/app.log` (all levels) and `logs/error.log` (errors and fatals). The format is plain text, not JSON:

```
2026-07-14T09:12:33.104Z LOG [Bootstrap] DB connected: true
2026-07-14T09:12:41.882Z LOG [RequestLoggerMiddleware] GET /api/v1/items/list 200 - 41.3ms
```

MEDIUM — LOGS ARE UNBOUNDED AND UNJOINABLE

There is no rotation. No size cap, no retention, no `logrotate` integration — the streams are opened once and appended to forever. `logs/app.log` currently stands at **168 MB**. Every request writes a line synchronously from the request path. The project's own `plan.md` claims "app.log / error.log with rotation"; that is not implemented. (R9)

There is no correlation ID. No request id is generated, none is propagated, and none appears in log lines or error responses. Two log lines from the same request cannot be joined, and a user reporting an error has no handle to quote. For a system of this size this is the single highest-leverage observability fix. (R9)

Beyond the log files there is nothing: no metrics endpoint, no tracing, no alerting, and no health signal other than `GET /api/v1/health`, which probes the database with `SELECT 1` and Redis with a real `PING`, returning `ok` or `degraded` (503 when a dependency is down). It does not check memory, event-loop lag, pool saturation or queue depth.

12.2 Load testing

`scripts/perf/load-test.js` is a dependency-free harness (829 lines) with three scenarios — `smoke` (15 s, 5 concurrent, health only), `baseline` (60 s, 20 concurrent, an 8-route weighted mix) and `stress` (180 s, 80 concurrent). It reports throughput, traffic, and min/avg/max/p50/p90/p95/p99 latency using reservoir sampling, and exits non-zero if the failure rate exceeds the scenario's threshold.

THE RECORDED PERFORMANCE BASELINES ARE INVALID

All three runs stored under `logs/perf/` were executed with `--accept-status 429`, which counts a rate-limit rejection as a success. The stress run reports 780,434 requests at 4,335 req/s with a 100% success rate — but **only 2,400 of those responses were HTTP 200**. The other 778,034 were 429s. The numbers measure how fast `ThrottlerGuard` can reject traffic, not how fast the application can serve it.

RUN	REQUESTS	HTTP 200	HTTP 429	REPORTED P95	ACTUAL COVERAGE
smoke	62,247	100	62,147	2.43 ms	0.16%
baseline	261,779	800	260,979	7.43 ms	0.31%
stress	780,434	2,400	778,034	24.59 ms	0.31%

The system's own `docs/performance-load-testing.md` §5 documents this trap and prescribes raising `THROTTLE_LIMIT` for capacity runs. **The platform therefore has no valid performance baseline.** Re-running the three scenarios with the throttler raised is a one-hour task and a prerequisite for any capacity claim. (R15)

12.3 Known performance characteristics

CHARACTERISTIC	DETAIL
Connection pools	Neither the Prisma client nor either <code>pg</code> pool sets an explicit connection limit; all three run on defaults. For a single-process deployment this is survivable, but it is untuned.
Pagination	Offset-based (<code>skip / take</code>) everywhere, including the configured-SQL engine, which wraps the base query in <code>COUNT(*)</code> plus a <code>LIMIT/OFFSET</code> subquery. Deep pages on large grids will degrade linearly.
Request timeout	A hard-coded 15,000 ms ceiling (<code>REQUEST_TIMEOUT_MS</code>) converts any longer handler into a 408. A slow configured grid will hit this rather than hang.
Concurrency	PM2 runs a single fork-mode instance — one Node process on one core. There is no clustering and no horizontal scale path today.
Synchronous I/O on the request path	Every request writes a log line to disk from the request path, and the audit trail adds an insert to every write transaction.

13 Build, packaging and deployment

13.1 The deployment model

The system is designed to be shipped as a **single self-contained executable** to hosts that have no Node runtime — the classic on-premise ERP distribution model. `npm run release` compiles with `nest build` and then runs `pkg`, producing `erp-server-linux` (204 MB) and `erp-server-win.exe` (196 MB).

The snapshot bundles the compiled output, the generated Prisma client and its native query engine, all 204 migration files, and — notably — the `.env` file itself. `src/main.ts` is written to support this: `resolveFilePath()` detects the `pkg` runtime and searches the working directory, then the executable's own directory, then the snapshot, so an operator can drop a TLS certificate or a replacement `.env` beside the binary.

LAYER	CONFIGURATION
Process manager	PM2, <code>ecosystem.config.js</code> : <code>instances: 1</code> , <code>fork mode</code> , <code>autorestart: true</code> , <code>PORT: 3012</code> . The <code>cwd</code> is a hard-coded developer absolute path and must be corrected before any real deployment.
Dependencies	<code>docker-compose.yml</code> provides Postgres 16 and Redis 7 only — there is no app service . The API runs on the host. Redis is published on 6379 with no password.
Reverse proxy	Not in this repository (an nginx config exists one level up in the workspace).
Migrations	<code>npm run migration:run</code> → <code>prisma migrate deploy</code> . The schema is rebuilt from fragments first, always.

TWO PACKAGING HAZARDS

The `.env` is baked into the binary. It is correctly git-ignored, but `pkg.assets` lists `".env"`, so the shipped executable embeds `DB_PASSWORD` and `JWT_SECRET` in its snapshot, and falls back to that copy when no external `.env` is found. Anyone holding the binary holds the secrets. Remove it from `assets` and require an external file.

The Windows target may have no query engine. Only `libquery_engine-debian-openssl-3.0.x.so.node` is listed in the bundled assets; there is no `query_engine-windows.dll.node`. The Windows build must fall back to the WASM/library engine or it will fail at runtime. This should be verified on a clean Windows host before shipping.

14 Quality engineering

14.1 Test coverage

Fifty-six unit spec files and two end-to-end specs. The tests that exist are well-formed — plain Jest with hand-rolled doubles, constructing the class directly rather than booting a Nest testing module — but they are concentrated almost entirely in one module. Two-thirds of all specs test Inventory.

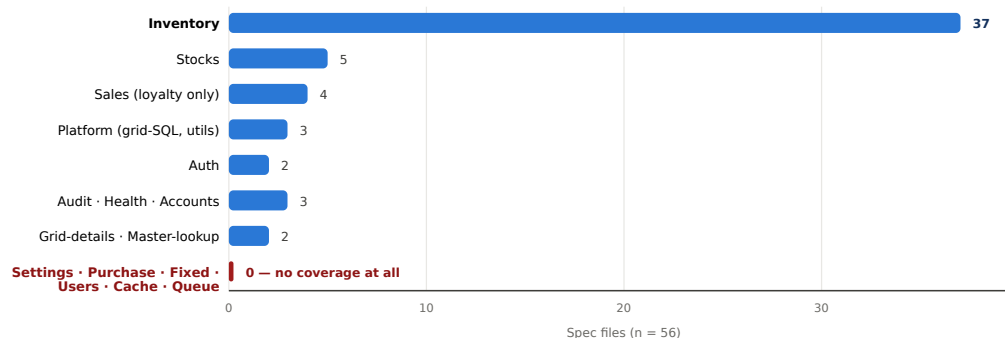


Figure 14.1 — Spec files by area. Whole subsystems — user administration, suppliers, reference data, the cache layer, the queues, the Redis client and PgService — have no tests.

14.2 Continuous integration

MEDIUM — THERE IS NO CI PIPELINE

`.github/` contains one file: `copilot-instructions.md`. There is no `.github/workflows/` directory and no CI configuration of any kind anywhere in the repository. Nothing runs lint, typecheck, tests or the build automatically. The npm scripts all exist (`lint`, `typecheck`, `test`, `test:cov`, `test:e2e`, `build`) and must be invoked by hand. There are also no coverage thresholds configured in Jest. The project's own README lists this as an open item. (R10)

The end-to-end suite boots the full `AppModule` through `Test.createTestingModule`, re-applies the validation pipe, versioning and prefix, and exercises `/api/v1/health` with supertest. It asserts a status of *either* 200 or 503 — that is, it tolerates a dead database — so it verifies wiring rather than behaviour.

15 Risk register

Every finding below was verified against the source at commit `5b3b3f0`. Severity reflects exploitability and blast radius, not effort to fix. Items R1–R5 should be treated as release-blocking.

ID	SEVERITY	FINDING	IMPACT AND LOCATION
R1	CRITICAL	Users controller is entirely unauthenticated	Class-level <code>@Public()</code> exposes user creation, enumeration and password change with no token. A complete authentication bypass: create a user, then log in legitimately. <code>users.controller.ts:36</code>
R2	CRITICAL	No authorisation and no tenant isolation	Role guards and the Prisma tenant-scope extension were removed in commit <code>80ab8eb</code> and never replaced. Any authenticated user reaches every controller and every company's rows. Menu permissions are enforced only by the UI. <code>app.module.ts:193</code>
R3	HIGH	Authenticated users can author server-executed SQL; dropdown parameters are interpolated, not bound	Any user may store <code>grid_sql</code> / <code>dropdown_sql</code> and execute it. <code>substituteGridPrm()</code> concatenates escaped literals. Confined to <code>SELECT</code> on a read-only session — but only if <code>DATABASE_READONLY_URL</code> is configured. <code>configured-grid-sql.service.ts:926</code> , <code>dropdown-details.service.ts:118</code>
R4	HIGH	Audit read API is unscoped	Any authenticated user reads all tenants' before/after snapshots, which contain PAN, Aadhaar, GSTIN, bank accounts and salary. <code>audit-log.controller.ts:30</code>
R5	HIGH	Session revocation fails open	Every <code>AuthServiceService</code> method returns early when Redis is disabled or unreachable. A valid signature alone then grants access, and revocation silently does nothing. <code>auth-session.service.ts:26</code>
R6	MEDIUM	No brute-force protection on login	No login-specific throttle and no logout — <code>usrFailedLoginCount</code> is incremented but <code>usrIsLocked</code> is never written anywhere. ~100 guesses/min/IP. Throttler storage is in-memory, so the limit is per-process. <code>auth.service.ts:239</code>
R7	MEDIUM	HTTP cache key has no user or tenant dimension	Cached <code>GET</code> responses are shared across all callers. Currently low-impact because TTLs are ~1 s and cached data is largely tenant-agnostic; becomes a cross-tenant leak the moment a scoped endpoint is cached. <code>http-cache.interceptor.ts:79</code>
R8	MEDIUM	Cache TTL unit bug	<code>ttlSeconds * 1</code> is passed to a millisecond-based store: <code>REDIS_TTL=3600</code> yields 3.6 s. Combined with the near-universal <code>@CacheTTL(1)</code> , the cache tier delivers almost none of its intended benefit. <code>redis.module.ts:28</code>
R9	MEDIUM	No log rotation; no correlation ID	<code>app.log</code> is at 168 MB and grows without bound; every request writes synchronously. No request id exists anywhere, so log lines cannot be joined to a request and users cannot quote an error handle. <code>file-logger.service.ts</code>
R10	MEDIUM	No CI; uneven test coverage	No workflow runs lint, typecheck, tests or build. No coverage thresholds. Settings, purchase, fixed, users, cache, queue and <code>PgService</code> have zero specs. <code>.github/</code>
R11	MEDIUM	Swagger exposed in production; CSP allows inline scripts	The full API surface, including schemas, is published unauthenticated. <code>'unsafe-inline'</code> in <code>scriptSrc</code> applies to every route, not just the docs. <code>main.ts:116,219</code>
R12	LOW	Two divergent error envelopes	Clients cannot parse errors uniformly; <code>message</code> may be a string, an array or a nested object depending on the route. §6.2
R13	LOW	Single-process deployment; untuned pools	PM2 fork mode, <code>instances: 1</code> ; no connection limits set on Prisma or either <code>pq</code> pool; hard-coded developer <code>cwd</code> in the PM2 config. <code>ecosystem.config.js</code>
R14	LOW	Dead and dormant code	Two BullMQ queues with no producers; an orphaned duplicate <code>item-price-details</code> module; <code>.env</code> baked into the <code>pkg</code> binary; config defaults that disagree across three files (<code>THROTTLE_LIMIT</code> 100 vs 10000; <code>PORT</code> 3000 vs 3010 vs 3012).
R15	LOW	Performance baselines are invalid	All recorded runs accepted HTTP 429 as success; 99.7% of responses were rate-limit rejections. The platform has no valid capacity baseline. <code>logs/perf/*.json</code>

16 Remediation roadmap

Sequenced by dependency and risk, not by module. P0 is the set of changes without which the system should not be exposed to untrusted networks; each is small and well-localised.

16.1 P0 — before any production exposure

REF	ACTION	NOTES
R1	Remove the class-level <code>@Public()</code> from <code>UserController</code>	One line. Then decide deliberately which, if any, user routes may remain public — probably none. Verify no client depends on the current behaviour.
R2	Reinstate authorisation and tenant scoping	The deleted <code>RolesGuard</code> and <code>tenant-scope.extension.ts</code> are recoverable from commit <code>80ab8eb</code> . Restoring the Prisma extension is the highest-leverage single change in this document: it re-scopes ~35 models on every read in one place, rather than auditing 87 services. Remove the stale comment in <code>items-master.service.ts:602</code> either way.
R3	Gate grid/dropdown SQL authoring behind an admin role; set <code>DATABASE_READONLY_URL</code>	Depends on R2. Configuring a genuine read-only Postgres role is a five-minute change that converts the last line of defence from advisory to physical.
R4	Scope the audit read API by company and branch	Add the predicate from the request context, and consider restricting it to an auditor role.
R5	Make session assertion fail closed	If Redis is <i>configured</i> but unreachable, reject rather than pass. Keep the fail-open path only for a deliberate <code>REDIS_ENABLED=false</code> deployment, and log loudly at boot when revocation is inactive.
R6	Throttle <code>/auth/login</code> and enforce lockout	<code>@Throttle()</code> on the login handler, and wire <code>usrFailedLoginCount</code> to actually set <code>usrIsLocked</code> — the column already exists.

16.2 P1 — operational readiness

- **Introduce a correlation ID** (R9). Generate one per request in `RequestContextMiddleware`, put it in the ALS store, include it in every log line and in both error envelopes, and echo it as `X-Request-Id`. This is a half-day change that makes every future incident tractable.
- **Rotate logs** (R9) — size-capped rotation with retention, and move the request log off the synchronous request path.
- **Add a CI pipeline** (R10): lint, typecheck, unit tests, build, and a migration check on every pull request. Set a coverage floor and ratchet it.
- **Fix the cache tier** (R7, R8): correct the millisecond conversion, add the user/company dimension to the cache key, replace `KEYS` with `SCAN` in the invalidator, and set TTLs that reflect the comments' intent.
- **Gate Swagger** behind an environment flag or basic auth in production (R11), and scope `'unsafe-inline'` to the docs route.
- **Re-establish a performance baseline** (R15) with the throttler raised, and record it as the reference for future regressions.

16.3 P2 — architectural consolidation

- **Unify the error envelope** (R12) — a single global filter, one shape, `statusCode` and correlation id always present. Delete the 55 per-module filters, which currently differ only in a regular expression used to guess field names.
- **Migrate the dropdown path** to `bindGridParams` and delete `substituteGridPrm` (R3), so no code path concatenates values into SQL.
- **Resolve the queues** (R14): schedule the archival and reconciliation jobs, or remove them. Make the reconciliation processor set-based rather than row-by-row.

- **Extend test coverage** to the configured-SQL engine, the auth guard, the cache interceptors and the stock ledger — the four subsystems where a silent defect is most costly.
- **Housekeeping:** delete the orphaned `item-price-details` module; remove `.env` from `pkg.assets`; reconcile the three conflicting default values for `THROTTLE_LIMIT` and `PORT`; fix the `prisma-docs-generator` output path; verify the Windows query engine.

CLOSING ASSESSMENT

The domain engineering here is strong and, in places, genuinely sophisticated — the stock ledger, the voucher numbering, the linked-ledger invariant and the configured-SQL engine are all the work of someone who understands the problem. The gaps are almost entirely in the surrounding envelope: authorisation, isolation, observability and automation. That is a fortunate shape for a codebase to have, because the envelope is where fixes are cheapest. The P0 list above is perhaps a week of focused work, and most of it is restoring code that already existed.

A Appendix — configuration surface

Forty environment variables are validated by Joi at boot (`src/config/env.validation.ts`). `JWT_SECRET` is required with a minimum length of 16 outside `NODE_ENV=test`; a missing or short value aborts the boot. Note that `API_VERSION` is consumed on every route but is *not* in the Joi schema.

GROUP	VARIABLES
Application	<code>NODE_ENV</code> , <code>HOST</code> , <code>PORT</code> , <code>API_PREFIX</code> , <code>API_VERSION</code> *, <code>REQUEST_BODY_LIMIT</code>
Transport	<code>CORS_ORIGINS</code> , <code>CORS_CREDENTIALS</code> , <code>HTTPS_ENABLED</code> , <code>HTTPS_CERT_PATH</code> , <code>HTTPS_KEY_PATH</code> , <code>HTTPS_PASSPHRASE</code>
Database	<code>DATABASE_URL</code> , <code>DATABASE_READONLY_URL</code> , <code>DB_HOST</code> , <code>DB_PORT</code> , <code>DB_USER</code> , <code>DB_PASSWORD</code> , <code>DB_NAME</code> , <code>DB_SSL</code> , <code>DB_SYNC</code> , <code>DB_LOGGING</code>
Auth	<code>JWT_SECRET</code> , <code>ACCESS_TOKEN_TTL_SECONDS</code> , <code>REFRESH_TOKEN_TTL_SECONDS</code>
Rate limiting	<code>THROTTLE_ENABLED</code> , <code>THROTTLE_TTL</code> , <code>THROTTLE_LIMIT</code>
Redis	<code>REDIS_ENABLED</code> , <code>REDIS_URL</code> , <code>REDIS_HOST</code> , <code>REDIS_PORT</code> , <code>REDIS_TTL</code> , <code>REDIS_USERNAME</code> , <code>REDIS_PASSWORD</code> , <code>REDIS_DB</code> , <code>REDIS_TLS</code> , <code>REDIS_CONNECT_TIMEOUT_MS</code>
Logging	<code>LOG_FILE_PATH</code> , <code>ERROR_LOG_FILE_PATH</code>

* consumed but not validated. `DB_USER`, `DB_PASSWORD` and `DB_NAME` carry defaults in both Joi and `configuration.ts`, so a misconfigured deployment falls back silently instead of failing fast.

B Appendix — route index

Sixty-nine controllers. All routes are prefixed `/api/v1/`.

DOMAIN	CONTROLLER PREFIXES
Inventory	items, item-groups, item-categories, item-brands, item-sections, units, item-unit-conversions, item-gst-units, item-ean-codes, item-prices, item-qtywise-rates, item-reorders, item-taxes, item-tax-histories, item-price-details, godowns
Sales	customers, customer-groups, states, cities, areas, quotations, freight-charges, promotion-loyalty-points
Accounts	account-groups, account-ledger-masters, ledger-bank-accounts, ledger-shipping-addresses, tender-masters, tender-type-masters, company-group-masters, gsp-provider-masters
Stocks	opening-stocks, physical-stock, item-stock-balance, item-batch-stock
Settings	company-masters, branch-masters, employee-masters, employee-department-masters, employee-designation-masters, user-administration, configs, gsp-company-services
Fixed	bank-lists, hsn-code-masters, state-code-masters, device-list-masters, menu-masters, price-level-masters, stock-adj-reasons, ui-table-masters, user-login-sessions
Purchase	suppliers, supplier-groups
Platform	auth, users, health, audit-logs, master-lookups, grid-details, dropdown-details, configured-grid-sql, sequences, item-price-lookup, item-cust-rates, batch-prefixes, widget-masters

C Appendix — glossary

Configured SQL	A <code>SELECT</code> statement stored in <code>grid_details.grid_sql</code> or <code>dropdown_details.dropdown_sql</code> , validated and executed at runtime on a read-only pool. The mechanism by which screens are data, not code.
Linked ledger	The invariant that a customer or supplier shares its primary key with an <code>acc_ledger_master</code> row. The ledger is created first; its <code>led_id</code> becomes the master's id.
Godown	Warehouse or storage location (Indian ERP usage). Stock balances are scoped by godown.
Voucher	An accounting document. Every stock document produces a voucher header, which is how stock movements reach the ledger.
Toggle delete	The house delete idiom: <code>DELETE</code> flips the <code>*_is_deleted</code> flag and returns <code>{id, deleted: boolean}</code> . Nothing is hard-deleted.
Period key	The reset horizon of a document-number sequence — fiscal year, month, or <code>ALL</code> . Part of the sequence's uniqueness scope.
Request context	An <code>AsyncLocalStorage</code> store holding <code>ipAddress</code> , <code>userId</code> , <code>userType</code> and <code>companyId</code> , populated by the access-token guard from the verified token — never from headers.
p_ parameter	A named placeholder in configured SQL (<code>p_comp_id</code> , <code>p_branch_id</code>). The <code>p_</code> prefix exists so a parameter token can never collide with a real column name.

End of document. ERP Server — Backend Architecture Dossier, revision 1.0, 14 July 2026. Verified against branch `DEV-v1` at commit `5b3b3f0`. Confidential — internal use only.